Aprendizaje Automático - Libro 2 Clase 8 a 9

2. Clase 9

Clase Nº 9: Aprendizaje No Supervisado (III)

Contenido: Análisis de componentes principales.

En la clase de hoy trabajaremos los siguientes temas:

- Análisis de componentes principales (PCA)
  - Objetivo de PCA
  - Proceso de PCA
  - Aplicaciones de PCA
  - Implementar PCA con scikit-learn

1. Presentación:

¡Bienvenidos a nuestra clase N° 9 de Aprendizaje Automático y, en particular, al emocionante mundo del Análisis de Componentes Principales (PCA)! En esta sesión, exploraremos a fondo esta poderosa técnica de reducción de dimensionalidad que desempeña un papel crucial en la simplificación y comprensión de datos complejos. A lo largo de la clase, desentrañaremos los entresijos del PCA, desde la centralización de datos hasta la proyección en componentes principales. Además, exploraremos aplicaciones prácticas en una variedad de campos, desde la reducción de ruido en imágenes médicas para mejorar el diagnóstico de enfermedades, hasta la optimización de motores de recomendación en plataformas de streaming de música, y la identificación de patrones en datos financieros para la detección de fraudes. Prepárense para sumergirse en el fascinante mundo de PCA y descubrir cómo esta técnica puede impulsar la toma de decisiones inteligentes en una amplia gama de aplicaciones del mundo real.

2. Desarrollo y Actividades:

Análisis de componentes principales (PCA)

El Análisis de Componentes Principales, abreviado como PCA (Principal Component Analysis en inglés), es una técnica fundamental en el campo del aprendizaje automático, especialmente en el aprendizaje no supervisado. PCA es una técnica de reducción de dimensionalidad que busca transformar un conjunto de datos de alta dimensionalidad en un conjunto de datos de menor dimensionalidad, conservando al máximo la información relevante.

Objetivo de PCA

El objetivo principal de PCA es simplificar la representación de los datos, reduciendo la cantidad de variables (dimensiones) mientras se intenta retener la mayor cantidad de información posible. Esto facilita la visualización y comprensión de los datos, además de reducir el ruido y la redundancia en los mismos.

Proceso de PCA

El proceso de PCA implica los siguientes pasos:

- *Centrar los datos*: En este paso, se resta la media de cada característica a todas las muestras, lo que ayuda a centrar los datos en el origen.

- *Calcular la matriz de covarianza*: La matriz de covarianza se utiliza para comprender cómo las diferentes características se relacionan entre sí en términos de su variabilidad conjunta.

- *Calcular los autovectores y autovalores*: A través de la descomposición de la matriz de covarianza, se obtienen los autovectores y autovalores. Los autovectores representan las direcciones principales en las que se encuentra la máxima variabilidad, mientras que los autovalores indican la cantidad de varianza explicada por cada autovector.

- *Selección de componentes principales*: Se eligen los autovectores (componentes principales) que corresponden a los autovalores más grandes. Estos autovectores representan las direcciones en las que los datos tienen la mayor variabilidad.

- *Proyección de datos*: Finalmente, los datos originales se proyectan en el espacio de los componentes principales seleccionados, lo que da como resultado un nuevo conjunto de datos con menos dimensiones.

Aplicaciones de PCA

PCA tiene una amplia gama de aplicaciones en aprendizaje automático, como la reducción de ruido en datos, la visualización de datos en 2D o 3D, la eliminación de la multicolinealidad en modelos de regresión, y la extracción de características relevantes en problemas de reconocimiento de patrones.

Implementar PCA con scikit-learn

En scikit-learn, PCA se implementa como un objeto transformador que aprende componentes en su método de ajuste, y se pueden utilizar en nuevos datos para proyectarlos en estos componentes.

Podemos ver cómo trabaja, si cargamos el siguiente módulo:

```
import mglearn
```

Antes de mostrar el código para poder ejecutar este ejemplo debemos verificar si el paquete *mglearn* se encuentra instalado en nuestro ambiente:

```
# Se verifica si esta instalado el paquete
pip show mglearn
```

En caso que sea necesario debemos instalarlo y reiniciar el kernel:

```
# Se instala el paquete
pip install mglearn
```

Y ejecutamos:

```
mglearn.plots.plot_pca_illustration()
```

Esto nos despliega una serie de gráficos, en donde en el primero se muestra una representación en dos dimensiones de dos características (*feature2* vs *feature1*). En el gráfico vemos cómo a partir de la muestra, *PCA* toma la dirección en donde existe una mayor variación de datos (mayor varianza) y traza un eje que denomina “componente 1” el cual es tomado como el componente principal, y luego toma el eje ortogonal (90°) y traza el “componente 2”, finalmente muestra una representación de los ejes sobre la distribución de puntos. Ambos ejes se trazan de forma tal de maximizar la varianza.

Vemos cómo se implementa:

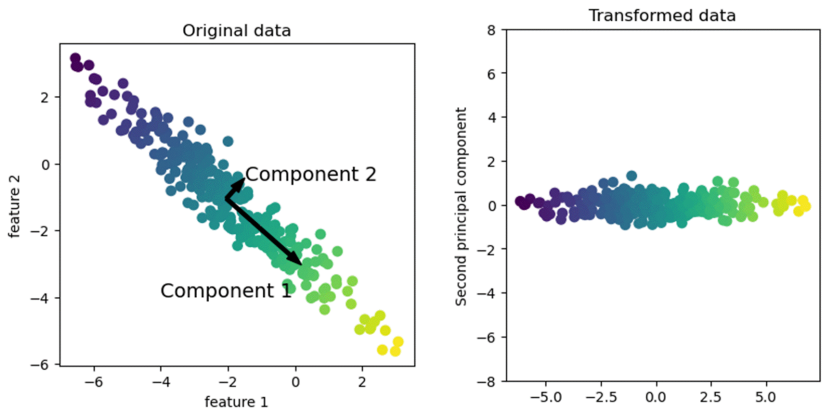
```
import sklearn
import mglearn
from sklearn import datasets
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
iris = datasets.load_iris()
print(iris.feature_names)

datos = iris.data
mglearn.plots.plot_pca_illustration()

pca=PCA(n_components=2)
pca.fit(iris.data)

transformada=pca.transform(datos)
print(iris.data.shape)
print(transformada.shape)
```

Resultando

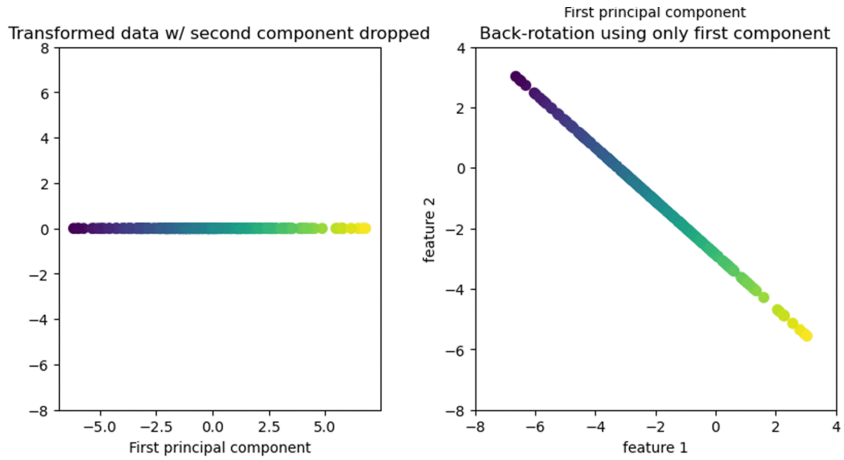


A partir de los dos ejes hallados intenta graficar todos los puntos y realiza una representación abstracta de los datos. En los gráficos anteriores se puede ver:

- En el *gráfico 1*, el trazado de los nuevos ejes.
- En el *gráfico 2*, la nueva asignación de ejes, lo cual causa una rotación de la representación, en donde a cada valor se le ha restado el promedio de datos para centrar los valores en cero tanto en el eje x como en el eje y.

También nos muestra un par de gráficos más en donde podemos ver:

- En el *gráfico 3*, la proyección de los datos sobre el nuevo eje correspondiente al componente principal.
- En el *gráfico 4*, elimina el ruido de los datos, el gráfico se ve con la misma dirección al gráfico 1 pero representado como una línea recta.



Utilizamos PCA para reducir el número de dimensiones que tenemos en el set de datos, indicando que realice la representación en dos ejes, y luego le cargamos los datos de iris.

```
pca=PCA(n_components=2)
pca.fit(iris.data)
```

Podemos ver que nuestros datos originales parten con un set de 150 datos y cuatro características y que luego tenemos 150 datos y dos características:

```
print(iris.data.shape)
print(transformada.shape)
```

Resultado:

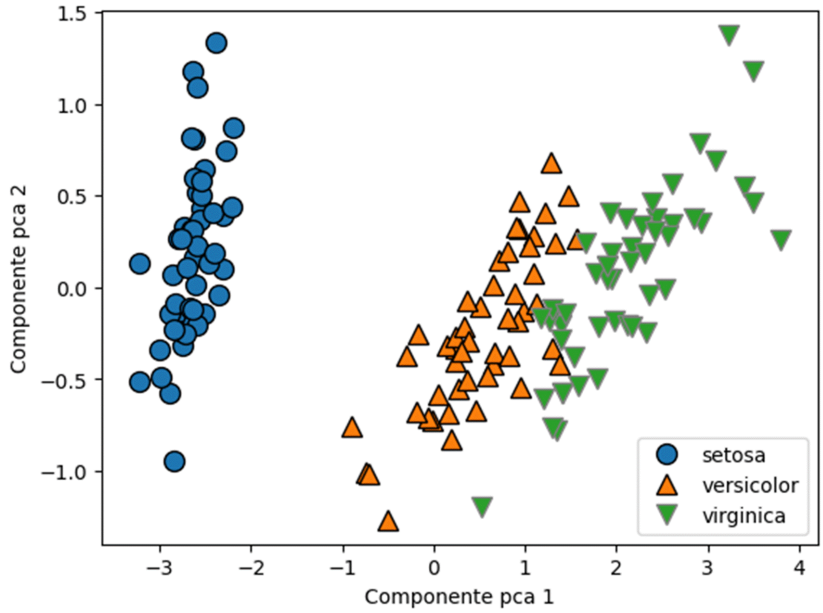
(150, 4)

(150, 2)

Al ejecutar

```
mglearn.discrete_scatter(transformada[:,0], transformada[:,1], iris.target)
plt.legend(iris.target_names, loc='lower right')
plt.xlabel('Componente pca 1')
plt.ylabel('Componente pca 2')
```

Podemos ver como el modelo realiza una clasificación de los datos en tres grupos.



Podríamos seguir la recomendación de tomar estos datos como entrada de *K-means* y ver el resultado obtenido:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn import datasets
from sklearn import metrics
iris = datasets.load_iris()
datos = iris.data
etiquetas = iris.target

k_means = KMeans(n_clusters=3, max_iter=2000)
k_means.fit(transformada)

predicciones=k_means.predict(transformada)

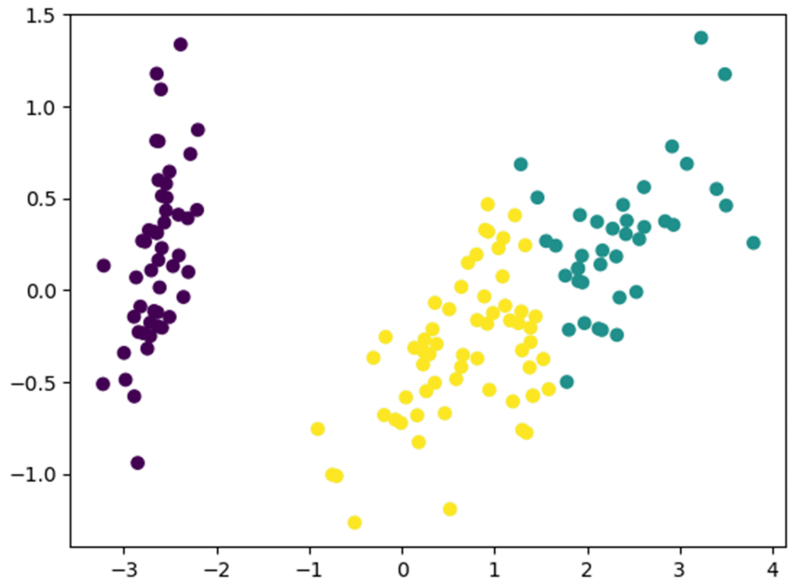
score=metrics.adjusted_rand_score(etiquetas, predicciones)

print(score)
plt.scatter(transformada[:, 0], transformada[:, 1], c=predicciones)

plt.show()
```

El resultado nos muestra un 71,6% de aciertos, no evidenciándose en este caso una mejora en el método de predicción.

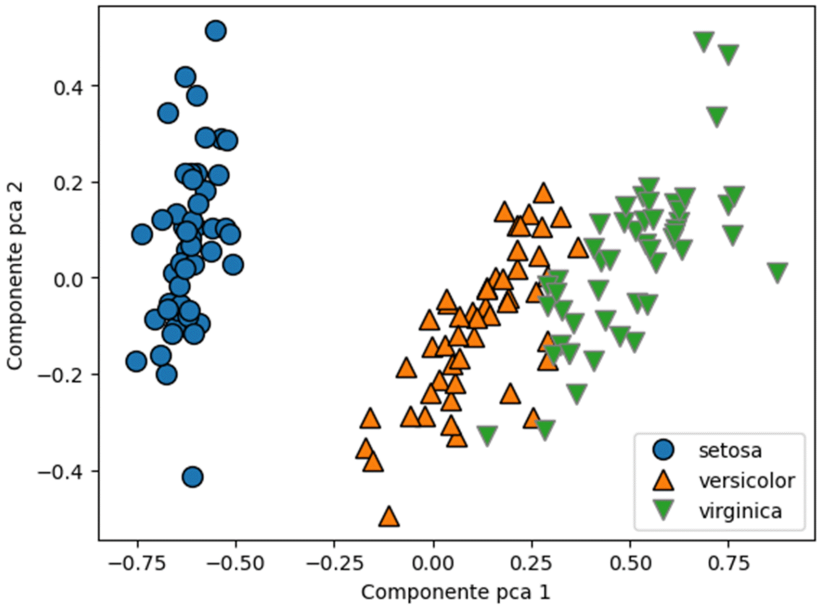
```
0.7163421126838476
```



Si ahora aplicamos el proceso de escalado que vimos en la clase pasada a nuestro trabajo, obtenemos:

```
from sklearn.preprocessing import MinMaxScaler
escala=MinMaxScaler()
escala.fit(iris.data)
escalada=escala.transform(iris.data)
pca.fit(escalada)
transformada=pca.transform(escalada)
mglearn.discrete_scatter(transformada[:,0], transformada[:,1], iris.target)
plt.legend(iris.target_names, loc='lower right')

plt.xlabel('Componente pca 1')
plt.ylabel('Componente pca 2')
```



En el caso del dataset iris no existe una diferencia apreciable en la representación de datos. Esto puede variar considerablemente en otros casos en donde la variación de valores es más apreciable.

3. Instancia de Autoevaluación: Participación en foro obligatorio

En esta instancia de autoevaluación tendrás que participar en el foro de la clase 9 respondiendo las siguientes preguntas ¿por qué resulta importante aplicar PCA a un conjunto de datos? ¿siempre resulta necesario hacerlo? ¿puede presentar algún problema aplicar PCA a un conjunto de datos? Buscá algún ejemplo de aplicación de PCA y compartilo en el foro.

4. Cierre:

En esta clase, hemos recorrido un emocionante viaje a través del Análisis de Componentes Principales (PCA) y otras técnicas clave de Aprendizaje Automático. Hemos aprendido cómo PCA puede transformar datos complejos en formas más manejables, y hemos reflexionado sobre su importancia en diversas aplicaciones, desde la mejora de diagnósticos médicos hasta la detección de fraudes en el sector financiero. Lo que hemos descubierto es que el conocimiento adquirido aquí no es solo una herramienta poderosa, sino una habilidad esencial para cualquier profesional que busque comprender y aprovechar el potencial oculto en los datos. En un mundo impulsado por la información, dominar estos conceptos es como tener una linterna en la oscuridad, permitiéndonos tomar decisiones informadas, descubrir patrones ocultos y encontrar soluciones efectivas en una amplia gama de campos. Así que, mientras cerramos esta clase, recordemos que el aprendizaje nunca se detiene, y que las habilidades que hemos adquirido aquí nos acompañarán en nuestra búsqueda constante de conocimiento y excelencia en el mundo del Aprendizaje Automático y más allá.

Te invitamos a que nos dejes tus impresiones sobre lo que aprendimos hoy en el foro. También podés compartir links de artículos o videos que te hayan parecido interesantes mientras estudiabas este tema.

Te esperamos en los próximos encuentros. Recordá que es importante participar de los foros, de los encuentros sincrónicos y en lo posible de las tutorías presenciales.

Webografía

"scikit-learn." *scikit-learn: machine learning in Python — scikit-learn 1.2.2 documentation*, https://scikit-learn.org/stable/. Accedido el 23 Junio del 2023.

5. Bibliografía Obligatoria:

- HAN, R. (2019): *Matemáticas del Aprendizaje Automático*. *Introducción a la analítica de datos e Inteligencia Artificial*.
- JONES, H. (2019): *Aprendizaje Automático. El Aprendizaje Automático para principiantes que desean comprender aplicaciones, Inteligencia Artificial, Minería de Datos, Big Data y más*. Editorial Edición Kindle.
- RUSSELL, R. (2018): *Machine Learning. Guia Paso a Paso Para Implementar Algoritmos De Machine Learning Con Python*
- VIDALES, A. (2019) *Machine Learning Con Matlab*. Editorial Edición Kindle.
- CANE, A. (2019): *Programación Con Python. Guía completa para principiantes.*
- HERNÁNDEZ, V. (2019): *Aprendizaje Automático. Implementación en Inteligencia Artificial.* Editorial Edición Kindle.